Otimizador Lookahead

Marcelo Heluy e Samuel Magalhães

Introdução - Otimizadores

- Treinamento de Redes Neurais como problema de otimização
  - Dado conjunto de dados e função de perda, queremos encontrar o conjunto de pesos que minimiza o erro de predição do modelo sobre exemplos não vistos.
- Algoritmo que percorre o espaço de parâmetros em busca de um ponto mínimo da função de perda.
  - Tipicamente não-convexa, de alta dimensionalidade, cheia de pontos de sela, platôs e mínimos locais. Escolha inadequada do otimizador leva a convergência prematura, oscilações ou falha no aprendizado do modelo.
- Gradiente Descendente Estocástico (SGD), Adam, Adagrad e RMSProp

Introdução - Otimizador Lookahead

Zhang et al. 2019

- Encapsula um algoritmo existente, mantendo dois conjuntos de peso: rápidos (θ) e lentos (φ), atualizando-os de forma interpolada a cada k passos do otimizador base. Pesos rápidos exploram o espaço de parâmetros com mais liberdade, enquanto os lentos âncoram o processo, evitando oscilações e melhorando estabilidade do treinamento.
- Tem bons resultados quando encapsula otimizadores clássicos como SGD e Adam, mas é igualmente eficaz com otimizadores com falhas estruturais conhecidas, como Adagrad (decaimento prematuro do aprendizado) e RMSProp (ausência de correção de viés)?

Trabalhos relacionados

- Ruder 2017, Kashyap 2023: apresentam variantes e aprofundam a análise do SGD, destacando dependência do algoritmo à taxa de aprendizado, tamanho do lote e convergência dos gradientes;
- Polyak 1964: Introdução do momentum - Acúmulo de médias móveis exponenciais, permitindo maior velocidade em direções consistentes e amortecimento em direções de curvatura.
- Duchi et al. 2011: Método adaptativo Adagrad - taxas de aprendizado individuais por parâmetro, beneficiando parâmetros com gradientes esparsos, mas gerando decaimento excessivo da taxa de aprendizado em cenários não convexos;
- Tieleman e Hinton 2012: Método adaptativo RMSProp - Substituição da soma acumulada do Adagrad por uma média móvel exponencial dos gradientes, contornando o problema de decaimento prematuro, mas sem mecanismo de correção de viés, gerando estimativas imprecisas nos passos iniciais do treinamento;
- Kingma e Ba 2012: Método Adaptativo Adam - Combinação do momentum e do RMSProp, aplicando médias móveis e correção de viés. Robusto e com baixo custo de ajuste de hiperparâmetros;
- Wilson et al. 2017: Soluções encontradas por métodos adaptativos generalizam pior do que as obtidas pelo SGD com momentum;

Fundamentação Teórica - Otimização de Redes Neurais

Treinamento de uma Rede Neural é a minimização do risco empírico sobre um conjunto de treinamento.

$$\theta^* = \text{argmin}(1/n) \sum_i L(f(x_i; \theta), y_i) + R(\theta)$$

- L: função de perda
- f(x;θ): predição do modelo
- R(θ): termo de regularização

$$\theta_{t+1} = \theta_t - \eta_t g_t$$

- η_t: taxa de aprendizado
- g_t: estimativa do gradiente

Fundamentação Teórica - Otimizadores Base

- SGD com momentum:

$$v_t = \beta v_{t-1} + \eta \nabla L(\theta_t), \beta \in [0, 1)$$

$$\theta_{t+1} = \theta_t - v_t$$

- Adagrad:

$$\theta_{t+1} = \theta_t - (\eta / \sqrt{(G_t + \varepsilon)}) \odot g_t$$

- RMSProp:

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

$$\theta_{t+1} = \theta_t - (\eta / \sqrt{(v_t + \varepsilon)})\, g_t$$

Fundamentação Teórica - Otimizadores Base

- Adam:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = m_t / (1-\beta)$$

$$\hat{v}_t = v_t / (1-\beta_t^2)$$

$$\theta_{t+1} = \theta_t - \alpha \left(\hat{m}_t / \sqrt{(\hat{v}_t + \varepsilon)} \right)$$

Fundamentação Teórica - Otimizador Lookahead

Para cada ciclo t = 1, 2, . . ., os pesos rápidos são sincronizados com os lentos ($\theta_{t,0} \leftarrow \varphi_{t-1}$) e, a seguir, o otimizador base executa k atualizações internas: $\theta_{t,i} \leftarrow \theta_{t,i-1} + A(L, \theta_t, i-1, d)$, onde d é um minibatch amostrado de D. Após os k passos internos, realiza-se a atualização externa dos pesos lentos:

$$\varphi_t = \varphi_{t-1} + \alpha(\theta_{t,k} - \varphi_{t-1}), \qquad \alpha \in (0, 1]$$

Fundamentação Teórica - Estado Interno do Otimizador Base

Há 3 estratégias para lidar com o estado interno e momentum do otimizador base após as atualizações dos pesos lentos.

- Manter o estado, sem modificações;
- Interpolar, aplicando a mesma interpolação linear dos pesos;
- Resetar, reiniciando o acumulador a cada k passos.

Metodologia - Objetivo e Hipóteses

Investigar empiricamente se o Lookahead é um regulador eficaz para os otimizadores com vulnerabilidades estruturais.

- Hipótese de Convergência: melhora a eficiência, reduzindo número de épocas necessárias;
- Hipótese de Estabilidade: reduz variância entre diferentes execuções;
- Hipótese de Mínimos Planos: guia o treinamento para mínimos mais planos.

Metodologia - Implementação do Encapsulador Lookahed

Lookahead foi implementado como uma classe Python, herdeira de torch.optim.Optimizer.

Parâmetros de entrada:

- otimizador base,
- número k passos,
- taxa α de aprendizado,
- estratégia de gerenciamento do momentum do otimizador base

Metodologia - Arquitetura e Dataset

Experimentos utilizaram fine-tuning da arquitetura ResNet-18 [He et al. 2016], com pesos pré-treinados no ImageNet.

Dataset utilizado: CIFAR-10

Pré-processamento consistiu em normalizar os valores de pixel para o intervalo [0,1] e redimensionamento das imagens 32x32 para 224x224 pixels via interpolação bilinear

Metodologia - Configuração dos Experimentos

Dois grupos:

- Otimizadores sem encapsulador
- Otimizadores com Lookahead

Otimizador	Taxa de aprendizado	
SGD	0,01	weight decay $5x10^{-4}$ momentum 0,9
Adam	0,001	-
Adagrad	0,01	-
RMSProp	0,001	$\alpha = 0,99$

- Lookahead aplicado com k=5 e α=0,5
- Treinamento com 10 épocas, com mini-batches de 64 imagens.
- Cada configuração foi executada com três seeds aleatórias (42, 123, 2024), fixadas na inicialização dos pesos da camada final e na ordem de amostragem.
- Execuções em ambiente Kaggle Notebooks utilizando duas GPUs NVIDIA Tesla T4.

Metodologia - Configuração dos Experimentos

Experimento adicional:

Avaliação do impacto da estratégia de gerenciamento do estado interno do otimizador base.

O SGD foi utilizado como otimizador base.

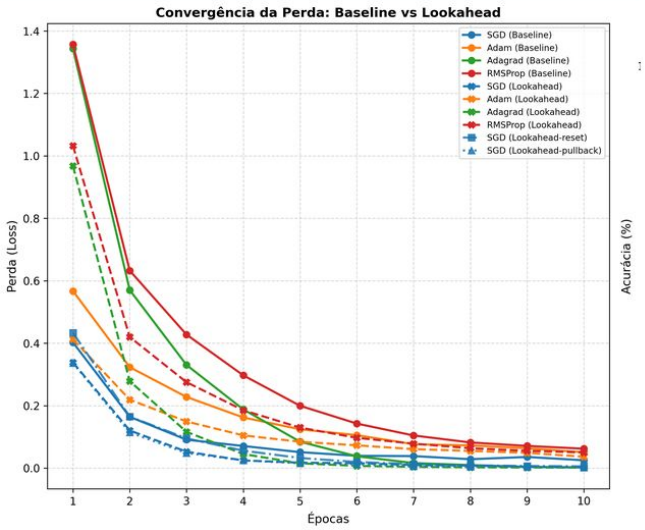
Resultados

Tabela 1. Acurácia final (%), média ± desvio padrão sobre 3 seeds (42, 123, 2024)

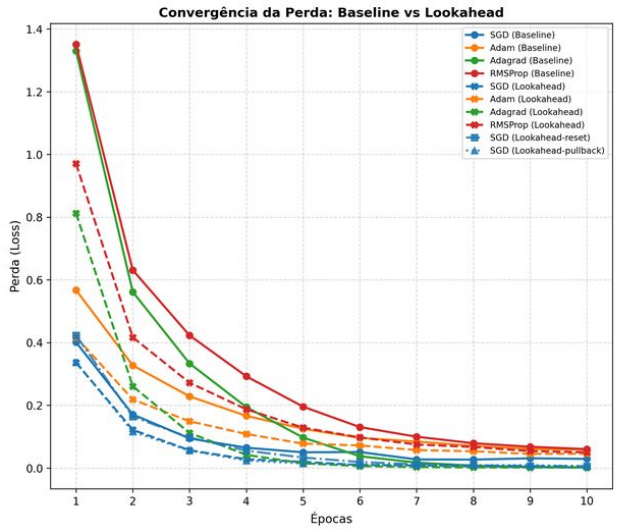
Otimizador	Configuração	Acurácia (%)
SGD	Baseline	$92,61 \pm 0,73$
	Lookahead (none)	$95,66 \pm 0,15$
	Lookahead (reset)	$95,05 \pm 0,04$
	Lookahead (pullback)	$95,31 \pm 0,42$
Adam	Baseline	$89,95 \pm 0,86$
	Lookahead (none)	$90,39 \pm 2,15$
Adagrad	Baseline	$88,90 \pm 3,32$
	Lookahead (none)	$86,13 \pm 13,52$
RMSProp	Baseline	$83,54 \pm 4,13$
	Lookahead (none)	$76,24 \pm 6,67$

Resultados

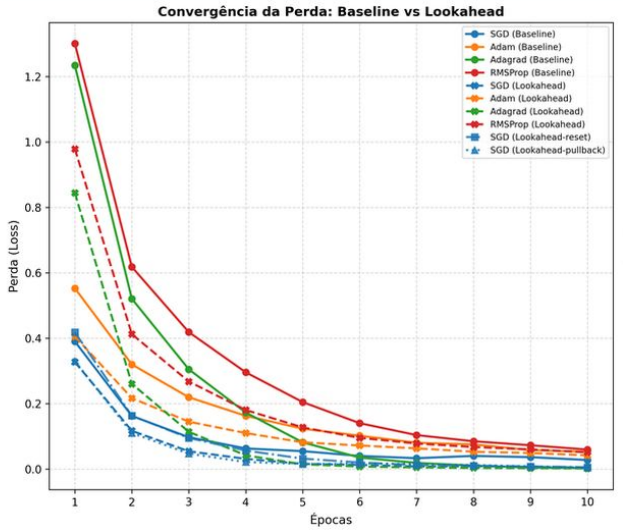
Seed 42



Seed 123

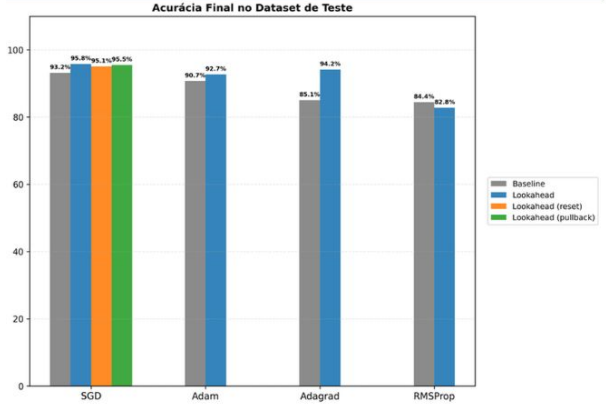


Seed 2024

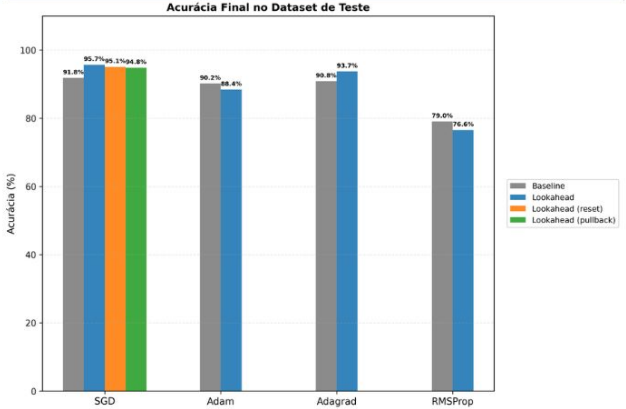


Resultados

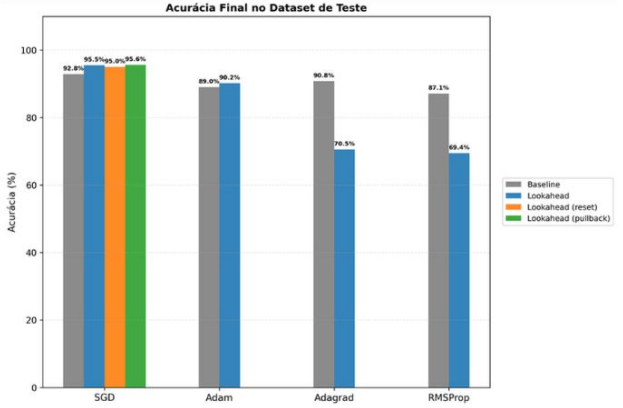
Seed 42



Seed 123



Seed 2024



Análise

- SGD: Ganho do Lookahead é robusto à estratégia de gerenciamento do estado interno.
- Adam: Comportamento mais estável entre os otimizadores adaptativos;

Adagrad e RMSProp

Melhora na acurácia nas seeds 42 e 123, mas perda significativa em 2024, consistente com cenário de overfitting ou de perda de generalização.

Hipótese plausível é a falha em gerenciar o momentum interno dos otimizadores. Essa dessincronia entre o estado dos pesos e o estado adaptativo do otimizador pode, dependendo da inicialização e da ordem de amostragem dos dados, levar o modelo a mínimos que minimizam bem a perda de treinamento mas generalizam mal.

Conclusão

Benefícios do Lookahead reportados por Zhang et al. 2019 não se estendem de forma homogênea a todos os otimizadores.

Os resultados com Adagrad e RMSProp reforçam a importância metodológica de executar multíplas seeds antes de generalizar conclusões.

Sugestão de trabalhos futuros:

- Investigação de estratégias de sincronização de estado interno do Adagrad e RMSProp
- Repetição dos experimentos com a configuração do artigo original.

Referências

Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. Journal of Machine Learning Research, 12:2121–2159.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778.

He, X., Xue, F., Ren, X., and You, Y. (2021). Large-scale deep learning optimizations: A comprehensive survey.

Hochreiter, S. and Schmidhuber, J. (1997). Flat minima. Neural Computation, 9(1):1–42.

Kashyap, R. V. (2023). A survey of deep learning optimizers – first and second order methods. Proceedings of the ACM on Measurement and Analysis of Computing Systems, 7(1).

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In International Conference on Learning Representations (ICLR).

Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. USSR Computational Mathematics and Mathematical Physics, 4(5):1–17.

Ruder, S. (2017). An overview of gradient descent optimization algorithms.

Tieleman, T. and Hinton, G. (2012). Lecture 6.5-rmsprop: Divide the gradient by a run- ning average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 4(2):26–31.

Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., and Recht, B. (2017). The margi- nal value of adaptive gradient methods in machine learning. In Advances in Neural Information Processing Systems (NeurIPS), volume 30.

Zagoruyko, S. and Komodakis, N. (2017). Wide residual networks.

Zhang, M. R., Lucas, J., Hinton, G., and Ba, J. (2019). Lookahead optimizer: k steps forward, 1 step back. In Advances in Neural Information Processing Systems, volume 32.